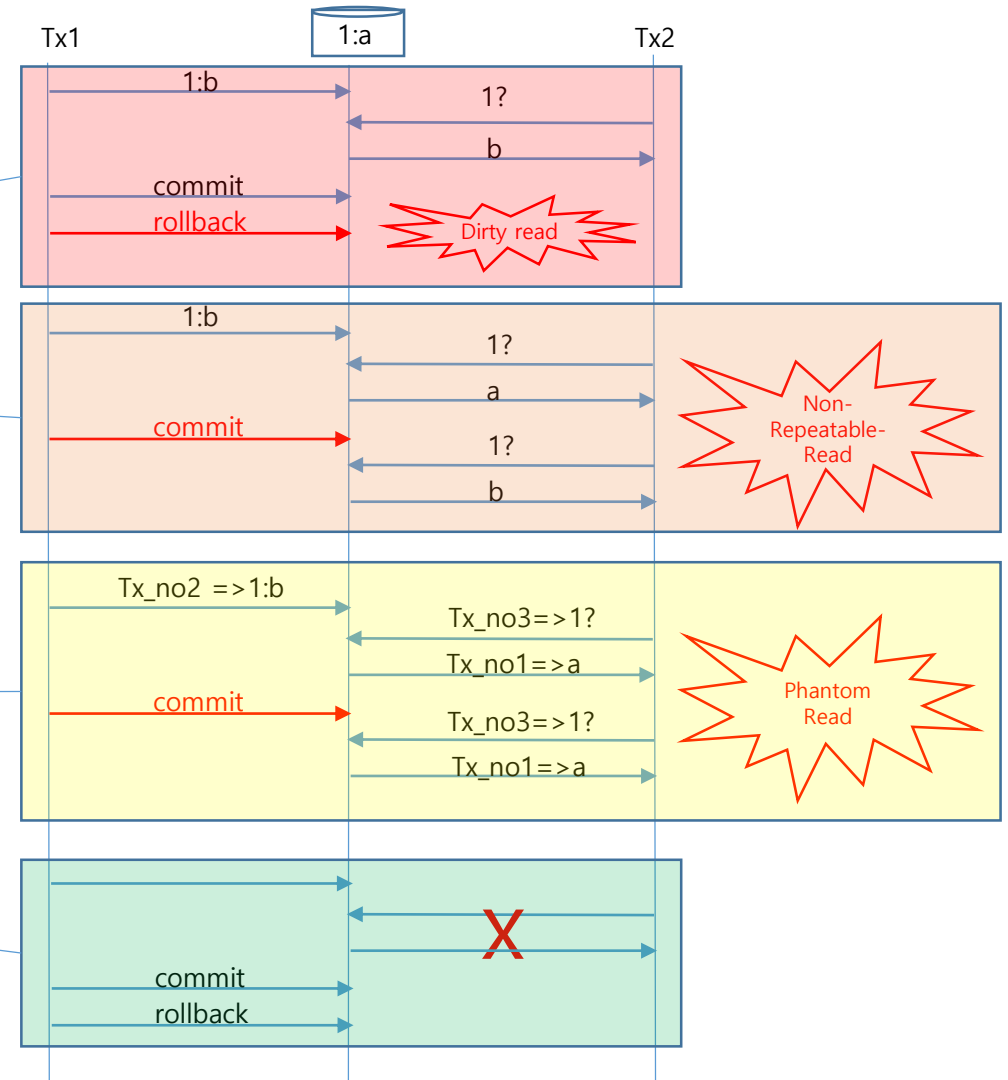


Transaction

- 원자성 (Atomicity) : 한번에 처리되어야 할 일의 단위
- 일관성 (Consistency) : 한 트랜잭션안에서 일관된 데이터 사용
- 독립성 (Isolation) : 다른 트랜잭션의 개입 불허
- 지속성 (Durability) : 결과 저장

옵션	설명
READ_UNCOMMITTED	트랜잭션에서 commit 되지 않은 다른 트랜잭션에서 읽는 것을 허용한다. Dirty Read가 발생한다. @Transactional(isolation = Isolation.READ_UNCOMMITTED)
READ_COMMITTED	트랜잭션에서 commit 되어 확정된 데이터만을 읽는 것을 허용한다. Oracle 기본 격리 수준이다. Non-Repeatable-Read 부정합이 발생할 수 있음 @Transactional(isolation = Isolation.READ_COMMITTED)
REPEATABLE READ	트랜잭션 시작 전 커밋된 내용만 조회 가능하다. MySQL InnoDB 기본 격리 수준이다. 한 트랜잭션 내에서 수 차례 SELECT 수행하더라도 동일한 값이 읽혀지는 것을 보장한다. Phantom Read 부정합이 발생할 수 있다. @Transactional(isolation = Isolation.REPEATABLE_READ)
SERIALIZABLE	모든 작업을 하나의 트랜잭션에 처리하는 것과 같은 높은 고립수준을 제공하는데, 이로 인해 동시성 처리 효율은 매우 떨어진다. @Transactional(isolation = Isolation.SERIALIZABLE)



	Dirty Read	Non-Repeatable Read	Phantom Read
READ_UNCOMMITTED	O	O	O
READ_COMMITTED	X	O	O
REPEATABLE_READ	X	X	O
SERIALIZABLE	X	X	X

옵션	설명	
REQUIRED	기본 옵션 부모 트랜잭션이 존재한다면 부모 트랜잭션에 합류, 그렇지 않다면 새로운 트랜잭션을 만든다. 중간에 자식/부모에서 rollback이 발생된다면 자식과 부모 모두 rollback 한다.	<pre> sequenceDiagram participant P as Parent participant C as Child P->>a(): a() P->>b(): b() C->>b(): b() C->>a(): a() P->>a(): a() </pre>
REQUIRES_NEW	무조건 새로운 트랜잭션을 만든다. nested한 방식으로 메소드 호출이 이루어지더라도 rollback은 각각 이루어 진다.	<pre> sequenceDiagram participant P as Parent participant C as Child P->>a(): a() P->>b(): b() C->>b(): b() C->>a(): a() P->>a(): a() </pre>
MANDATORY	무조건 부모 트랜잭션에 합류시킨다. 부모 트랜잭션이 존재하지 않는다면 예외를 발생시킨다.	<pre> sequenceDiagram participant P as Parent participant C as Child P->>a(): a() P->>b(): b() C->>b(): b() C-->>b(): b() C-->>a(): a() P-->>a(): a() </pre>
SUPPORTS	메소드가 트랜잭션을 필요로 하지는 않지만, 진행 중인 트랜잭션이 존재하면 트랜잭션을 사용한다는 것을 의미한다. 진행 중인 트랜잭션이 존재하지 않더라도 메소드는 정상적으로 동작한다.	<pre> sequenceDiagram participant P as Parent participant C as Child P->>a(): a() P->>b(): b() C->>b(): b() C->>a(): a() P->>a(): a() </pre>
NESTED	부모 트랜잭션이 존재하면 부모 트랜잭션에 중첩시키고, 부모 트랜잭션이 존재하지 않는다면 새로운 트랜잭션을 생성한다. 부모 트랜잭션에 예외가 발생하면 자식 트랜잭션도 rollback한다. 자식 트랜잭션에 예외가 발생하더라도 부모 트랜잭션은 rollback하지 않는다. 이때 롤백은 부모 트랜잭션에서 자식 트랜잭션을 호출하는 지점까지만 롤백된다. 이후 부모 트랜잭션에서 문제가 없으면 부모 트랜잭션은 끝까지 commit 된다.	<pre> sequenceDiagram participant P as Parent participant C as Child P->>a(): a() P->>b(): b() C->>b(): b() C->>a(): a() P->>a(): a() </pre>
NEVER	메소드가 트랜잭션을 필요로 하지 않는다. 만약 진행 중인 트랜잭션이 존재하면 익셉션이 발생한다.	<pre> sequenceDiagram participant P as Parent participant C as Child P->>a(): a() P->>b(): b() C->>b(): b() C-->>b(): b() C-->>a(): a() P-->>a(): a() </pre>